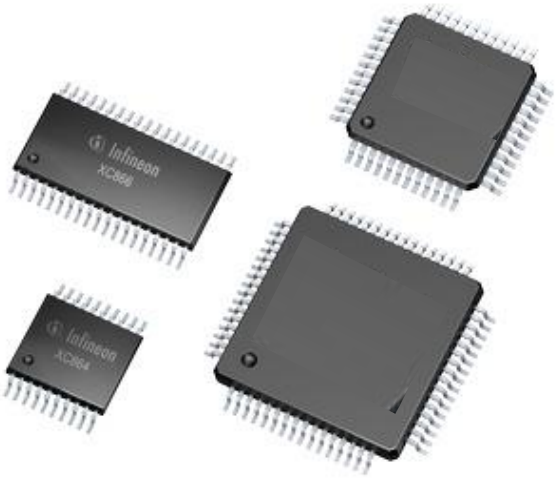
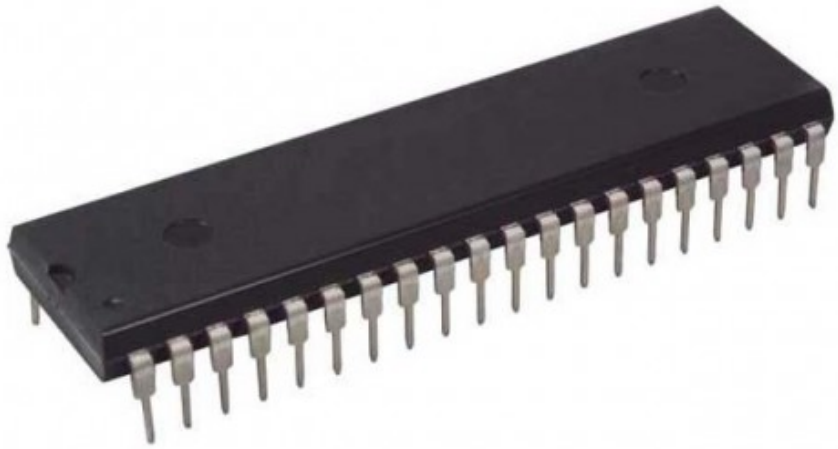
8051 Microcontroller





Difference between operation of PUSH and POP instruction?

PUSH: storing the CPU register in the stack

POP: Pulling the contents off the stack back into a CPU register

PUSH

8086	8051
SP 16 bit reg.	SP 8 bit reg.
SP decremented by 2	SP incremented by 1
SP=SP-2	SP=SP+1
Syntax: PUSH register Ex. PUSH AX	Syntax: PUSH address Ex. PUSH 6

POP

8086	8051
SP 16 bit reg.	SP 8 bit reg.
SP incremented by 2	SP decremented by 1
SP=SP+2	SP=SP-1
Syntax: POP register Ex. POP AX	Syntax: POP address Ex. POP 6

Register Bank 1= Default Stack memory

SP=07H

How to store data to Stack memory?

MOV R6, #25H; R6=25H

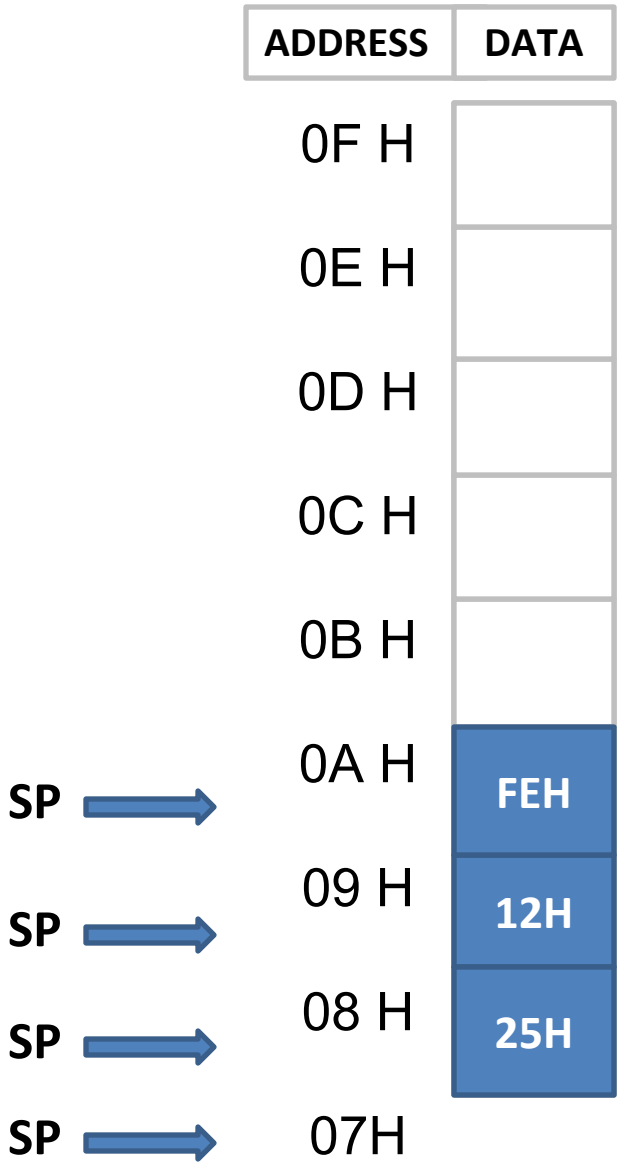
MOV R1, #12H; R1= 12H

MOV R4, #0FEH; R4=0FEH

PUSH 06h

PUSH 01h

PUSH 04h



Default position of stack pointer

Register Bank 1= Default Stack memory

SP=07H

How to store data to Stack memory?
CLR PSW.4

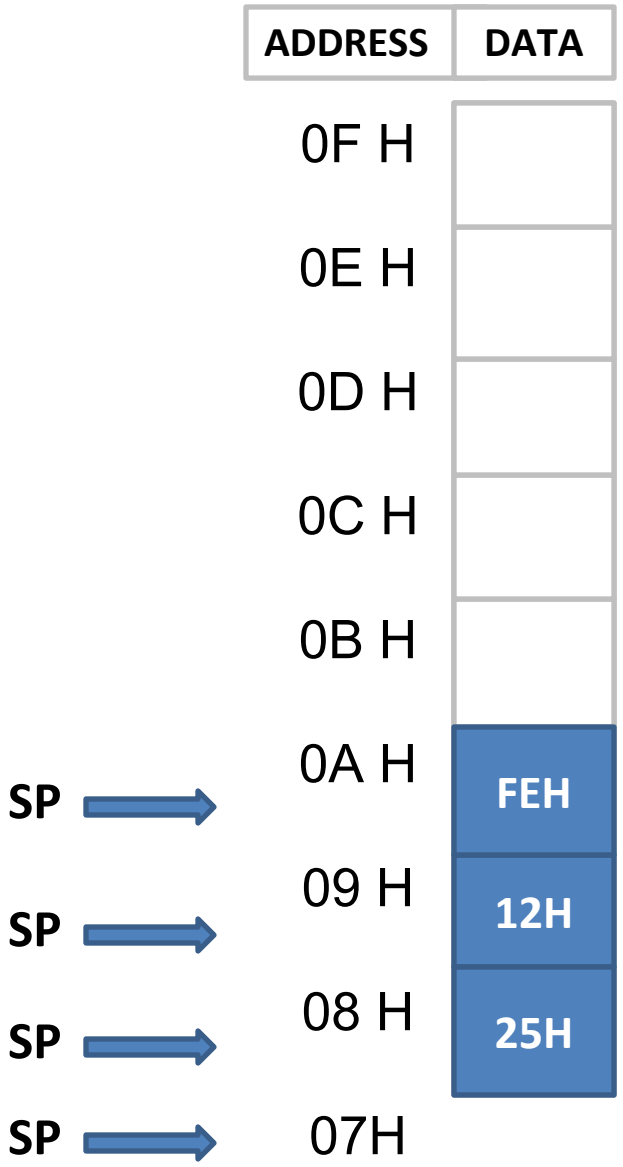
SETB PSW.3

MOV R6, #25H; R6=25H

MOV R1, #12H; R1= 12H

MOV R4, #0FEH; R4=0FEH

PUSH 0Eh
PUSH 09h
PUSH 0Ch



Default position of stack pointer

Register Bank 1= Default Stack memory

SP=07H

How to store data to Stack memory?

MOV 30h, #25H; R6=25H

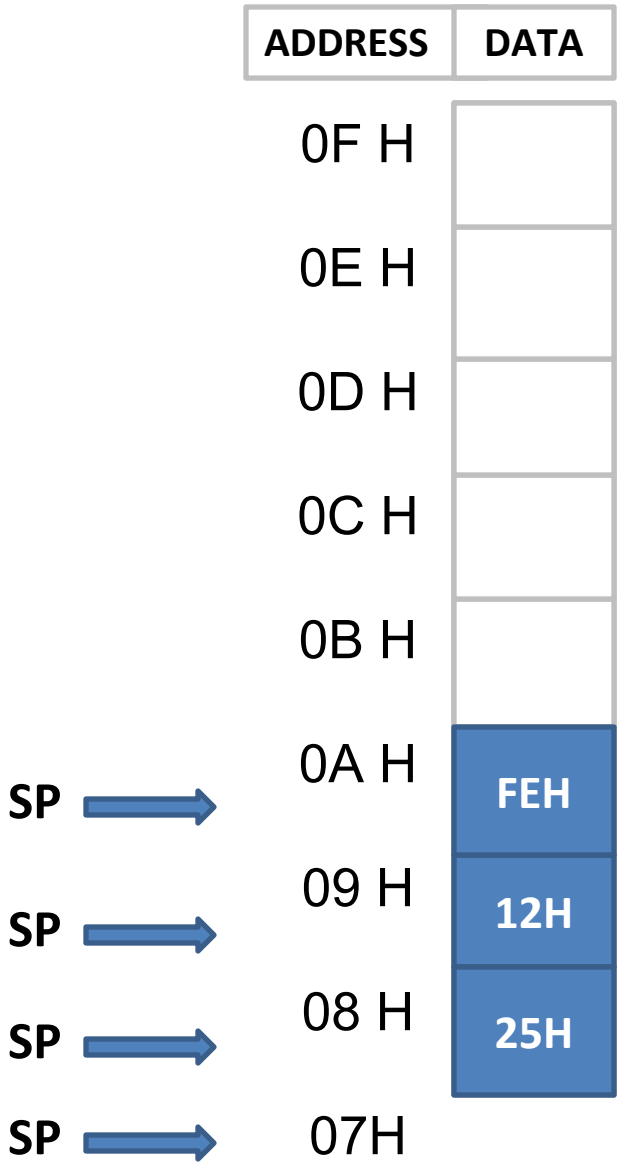
MOV 31h, #12H; R1= 12H

MOV 32h #0FEH; R4=0FEH

PUSH 30h

PUSH 31h

PUSH 32h



Default position of stack pointer

Register Bank 1= Default Stack memory
SP=07H

How to restore data from Stack memory?

MOV R6, #25H; R6=25H

MOV R1, #12H; R1= 12H

MOV R4, #0FEH; R4=0FEH

PUSH 6

PUSH 1

PUSH 4

POP 5; **R5=0FE**

POP 3; **R3=12H**

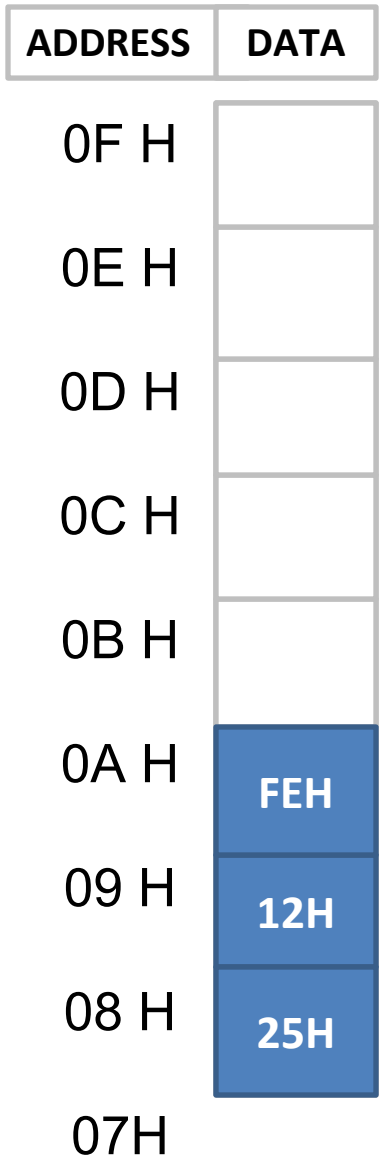
POP 2; **R2=25H**

Current status **SP** →

SP →

SP →

SP →



Register Bank 1= Default Stack memory
SP=07H

How to restore data from Stack memory?

```
ORG 00h

MOV R6, #25H; R6=25H

MOV R1, #12H; R1= 12H

MOV R4, #0FEH; R4=0FEH

PUSH 6

PUSH 1

PUSH 4

POP 10h
POP 11h
POP 12h

END
```

10h=0FE

11h=12

12h=25H

Current status **SP** →

SP →

SP →

SP →

ADDRESS	DATA
0F H	
0E H	
0D H	
0C H	
0B H	
0A H	FEH
09 H	12H
08 H	25H
07H	

LCALL and ACALL

Mnemonic	Description	Byte	Cycle
ACALL addr11	Absolute subroutine call	2	2
LCALL addr16	Long subroutine call	3	2
RET	Return from subroutine	1	2

LCALL

Long call

- 3 byte instruction, 1st byte opcode and 2nd, 3rd byte address
- Address: 16 bit

- LCALL can be used to call subroutines located anywhere within 2^16= 65536 address space of 8051

ACALL

Absolute call

- 2 byte instruction
- Address: 11 bit

- ACALL can be used to call subroutines located within 2048 address space of 8051

8051 microcontroller

0400h	MOV TMOD, #01H
0402h	**again1**: MOV TL0, #00H
0404h	MOV TH0, #0EEH
0406h	MOV P1, #0FFH
0408h	ACALL **DELAY**
040Ah	MOV P1, #00H
040Ch	ACALL **DELAY**
040Eh	SJMP **again1**

PC → 0408h

PC → 040Ah

Main code

PC → DELAY: SETB TR0

DELAY: SETB TR0
again: JNB TF0 again
CLR TR0
CLR TF0
RET

PC → RET

Sub-routine

PC=040Ah

Address RAM	Stack memory
0007h	0A
0008h	04
0009h	00

Address RAM	Stack memory
0007h	0A
0008h	04
0009h	00

LJMP

Long jump

- 3 byte instruction
- Address: 16 bit
- LJMP can be used to jump anywhere within 64 Kbyte address space of 8051

SJMP

Short jump

- 2 byte instruction
- 1st byte: opcode
- 2nd byte: relative address
- Address: 8 bit therefore relative address range= 00 to FFh
- Is divided into forward and backward jump, that is within -128 to 127 bytes of memory relative to address of current PC.

AJMP

Long jump

- 2 byte instruction
- Address: 11 bit
- AJMP can be used to jump anywhere within 2 Kbyte address space of 8051

PC	OPCODE		CODE
0002H	7455		MOV A,#55H
0004H	6003		JZ NEXT
0006H	08		INC R0
0007H	04		INC A
0008H	04		INC A
0009H	2477	NEXT:	ADD A, #77H

Relative address= target jump address- PC

PC	OPCODE		CODE
0002H	7455		MOV A,#55H
0004H	04	NEXT:	INC A
0005H	08		INC R0
0006H	04		INC A
0007H	60FB		JZ NEXT
0009H	2477		ADD A, #77H